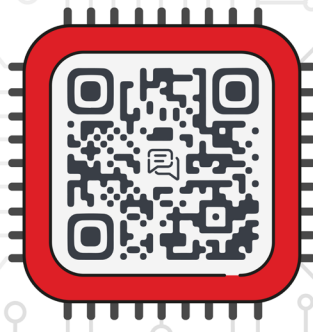




نظري

4



8



1200 sp

كلية الهندسة المعلوماتية

السنة الأولى

Casting & Object Relationships

د. فراس ضعيف
د. معتر النجرس



البرمجة 2

RB Informatcs; 03/05/2026

تمهيد ومراجعة سريعة

قبل الانتقال إلى مفهوم تحويل الأنماط، لا بد من التمييز بين مفهومين أساسيين في تعدد الأشكال داخل البرمجة كائنية التوجه:

- التحميل الزائد: (Overloading) وجود أكثر من تابع يحمل الاسم نفسه داخل الصنف نفسه، مع اختلاف في عدد البارامترات أو أنواعها أو ترتيبها. وقد يختلف نوع القيمة المعادة، لكنه لا يكفي وحده لتمييز تابع عن آخر في Java.
- إعادة الكتابة: (Overriding) وجود تابع في صنف الابن يطابق تابعاً موجوداً في صنف الأب من حيث الاسم والبارامترات ونوع القيمة المعادة، مع تقديم تنفيذ مختلف يناسب الصنف الابن.

تحويل الأنماط الأولية (Primitive Type Casting)

يقصد بمفهوم Casting تحويل قيمة من نمط بيانات إلى نمط آخر. وتنقسم هذه العملية في الأنماط الأولية، مثل int و float، إلى نوعين رئيسيين، بحسب اتجاه التحويل وإمكانية فقدان جزء من البيانات.

أولاً: التحويل التصاعدي التلقائي (Up-Casting / Implicit Casting)

يحدث التحويل التصاعدي عند نقل قيمة من نمط أصغر أو أضيق نطاقاً إلى نمط أوسع نطاقاً، مثل تحويل قيمة من int إلى float. تنفذ Java هذا النوع من التحويل تلقائياً دون الحاجة إلى كتابة التحويل صراحة.

• مثال:

```
int y = 10;
float x = y + 5;
```

في هذا المثال، يتم أولاً حساب التعبير $y + 5$ كعدد صحيح لأن كلا الطرفين من النوع int، ثم تحوّل النتيجة النهائية تلقائياً إلى float عند إسنادها إلى المتغير x. وبذلك تصبح القيمة المخزنة في x مساوية لـ 15.0.



ثانياً: التحويل التنازلي الصريح (Down-Casting / Explicit Casting)

يحدث التحويل التنازلي عند تحويل قيمة من نمط أوسع أو أكثر دقة إلى نمط أضيق، مثل تحويل قيمة من float إلى int. لا تنفذ Java هذا التحويل تلقائياً؛ لأن العملية قد تؤدي إلى فقدان جزء من البيانات، لذلك يجب كتابته صراحة.

- مثال:

```
float y = 1.5f;
int z = (int)y + 10;
```

- تحليل السلوك: عند تنفيذ التحويل الصريح (int)y، يتم اقتطاع الجزء العشري من القيمة، فتتحول 1.5 إلى 1، ثم تصبح قيمة z مساوية لـ:

1+10=11

1.5+10=11.5

بينما الناتج بعد التحويل هو 11، فإن التحويل التنازلي قد يؤدي إلى فقدان الدقة وإنتاج نتائج منطقية غير متوقعة. لذلك ينبغي استخدامه فقط عندما يكون فقدان الجزء العشري مقبولاً في منطق البرنامج.

تحويل أنماط الأصناف والتعددية (Reference Casting & Polymorphism)

كما يمكن تطبيق التحويل على الأنماط الأولية، يمكن أيضاً تطبيقه على المراجع الخاصة بالأصناف، بشرط وجود علاقة وراثية بين هذه الأصناف. ليكن لدينا الكود الآتي:

```
class Animal {
    public void makeSound() {
        System.out.println("Animal");
    }
}
class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Cat");
    }
    public void jump() {
        System.out.println("Jump");
    }
}
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Dog");
    }
}
```

```
public void run() {
    System.out.println("Run");
}
```

وقت التجميع مقابل وقت التنفيذ (Compile-time vs. Runtime)

عند كتابة السطر الآتي داخل الدالة الرئيسية main:

```
Animal a = new Cat(); //Up-casting
```

فإن المتغير المرجعي a مصرح عنه من نوع الأب Animal، بينما الكائن الفعلي المنشأ في الذاكرة هو من نوع الابن Cat.

- استدعاء التابع a.makeSound(); يعمل بنجاح ويطبوع:

Cat

- استدعاء التابع a.jump(); يؤدي إلى خطأ في وقت التجميع (Compilation Error)، ولا يقبل المترجم هذا الاستدعاء.

سبب خطأ التجميع:

- في وقت التجميع (Compile-time): يعتمد المترجم على نوع المتغير المرجعي، أي على النوع الموجود في الطرف الأيسر من الإسناد، وهو هنا Animal. وبما أن الصنف Animal لا يحتوي على تابع اسمه jump، فإن الاستدعاء يُرفض.
- في وقت التنفيذ (Runtime): يعتمد تنفيذ التوابع المعاد كتابتها على نوع الكائن الفعلي الموجود في الذاكرة، وهو هنا Cat.

القاعدة: التحقق من وجود التابع يتم في وقت التجميع اعتماداً على نوع المرجع، أما اختيار النسخة المنفذة من التابع المعاد كتابته فيتم في وقت التنفيذ اعتماداً على نوع الكائن الفعلي.

حل المشكلة باستخدام التحويل التنازلي للأصناف

لاستدعاء تابع خاص بالصنف الابن، مثل jump في الصنف Cat، لا بد من تحويل المرجع صراحة إلى نوع الصنف الابن: ((Cat) a).jump();
يكون هذا التحويل صحيحاً في هذه الحالة لأن الكائن الفعلي المشار إليه بالمرجع a هو فعلاً من نوع Cat.
أما إذا كان الكائن الفعلي ليس من نوع Cat، فإن التحويل قد يؤدي إلى خطأ أثناء التنفيذ من نوع ClassCastException. لذلك يُستخدم غالباً المعامل instanceof قبل تنفيذ التحويل التنازلي.



المصفوفات متعددة الأشكال والمعامل instanceof

تظهر فائدة التعددية الديناميكية بوضوح عند التعامل مع مصفوفة من نوع صنف الأب، بحيث يمكنها تخزين كائنات مختلفة من الأصناف الأبناء بناءً على مدخلات المستخدم أثناء وقت التشغيل.

```
public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Animal[] a = new Animal[10];

        for (int i = 0; i < 10; i++) {
            char c = input.next().charAt(0);

            if (c == 'c') {
                a[i] = new Cat();
            } else if (c == 'd') {
                a[i] = new Dog();
            } else {
                a[i] = new Animal();
            }
        }

        for (int i = 0; i < 10; i++) {
            a[i].makeSound();

            if (a[i] instanceof Cat) {
                ((Cat) a[i]).jump();
            } else if (a[i] instanceof Dog) {
                ((Dog) a[i]).run();
            }
        }
    }
}
```

في هذا المثال، صُرح عن المصفوفة على أنها مصفوفة من النوع Animal، لكنها قادرة على استقبال كائنات من Animal أو من أي صنف يرث منه، مثل Cat و Dog. عند تنفيذ السطر:

```
a[i].makeSound();
```

يتم اختيار النسخة المناسبة من التابع makeSound أثناء وقت التنفيذ، بحسب نوع الكائن الفعلي المخزن داخل الخانة، لا بحسب نوع المصفوفة فقط.

يُستخدم المعامل instanceof للتحقق مما إذا كان الكائن الحالي ينتمي إلى صنف معين، أو إلى صنف يرث منه، ضمن شجرة الوراثة. وتكون نتيجة هذا الفحص إما true أو false.

مثال ذلك أن الكائن المنشأ من الصنف Cat يحقق الشرطين الآتيين في الوقت نفسه:

```
a[i] instanceof Cat;
a[i] instanceof Animal;
```

والسبب أن القطعة تُعد حيواناً ضمن علاقة الوراثة، أي أن:

Cat is an Animal

بينما لا يصح عكس العلاقة دائماً، فلا يمكن القول إن كل كائن من نوع Animal هو بالضرورة Cat.

أنواع العلاقات بين الأصناف (Class Relationships)

تتفاعل الأصناف في التصميم كائني التوجه وفق أنماط مختلفة من العلاقات. ومن أهم هذه العلاقات نوعان رئيسيان:

أولاً: علاقة "هو نوع من" (Is-A Relationship)

تمثل هذه العلاقة مفهوم الوراثة (Inheritance)، حيث يكون الصنف الابن حالة خاصة أو أكثر تخصيصاً من الصنف الأب. مثال:

Cat is an Animal

أي أن القط نوع من الحيوانات. لكن العلاقة لا تعمل بالعكس، فلا يمكن القول إن كل حيوان هو قط. تُستخدم الوراثة عندما يكون الصنف الابن قابلاً للاستبدال منطقيًا مكان الصنف الأب، مع احتفاظه بالمعنى العام نفسه.

ثانياً: علاقة "يملك" أو "يحتوي على" (Has-A Relationship)

تظهر هذه العلاقة عندما يحتوي صنف ما على متغير مرجعي يمثل كائناً من صنف آخر. وفي هذه الحالة لا يكون الصنف المحتوي ابناً للصنف المحتوي، وإنما يكون جزءاً منه أو مرتبطاً به. مثال عام:

```
class Circle {
    private Point center;
}
```

في هذا المثال، الدائرة Circle تحتوي على نقطة Point تمثل مركزها. العلاقة هنا ليست وراثة، فلا يصح القول إن الدائرة هي نقطة، وإنما الصحيح أن الدائرة تملك نقطة مركز.

■ تنبيه هندسي حول الوراثة غير المناسبة:

لا يصح افتراض علاقة وراثة بمجرد وجود تشابه في الأسماء أو الخصائص. فمثلاً، جعل الصنف Point3D يرث مباشرة من الصنف Point قد يكون تصميمًا غير مناسب إذا كان التعامل الرياضي مع النقطة ثلاثية الأبعاد مختلفاً عن التعامل مع النقطة ثنائية الأبعاد. معيار صحة الوراثة لا يعتمد فقط على اشتراك الصنفين ببعض الحقول، بل يعتمد على صلاحية اعتبار الابن نوعاً حقيقياً من الأب وفق منطق البرنامج.

التركيب ضد التجميع والارتباط (Composition vs. Aggregation | Association)

تندرج علاقات الاحتواء تحت مفهوم عام هو علاقة Has-A، لكنها تختلف في قوتها بحسب درجة اعتماد الكائن المحتوى على الكائن المحتوي.

1. التركيب (Composition)

التركيب هو علاقة احتواء قوية، يكون فيها الكائن المحتوى جزءاً أساسياً من الكائن المحتوي، ولا يُنشأ عادةً بشكل مستقل عنه داخل التصميم. في هذه العلاقة، ترتبط دورة حياة الجزء بدورة حياة الكل. إذا اختفى الكائن المحتوي، فإن الكائنات التي أنشأها داخله لا يبقى لها وجود مستقل منطقي ضمن البرنامج.

- مثال برمجي: صنف الدائرة Circle يحتوي داخلياً على نقطة المركز Point center. في هذا التصميم، تنشأ نقطة المركز داخل باني الدائرة، فتكون مرتبطة بها ارتباطاً قوياً.

```
class Point {
    public int x, y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

class Circle {
    private Point center;
    int r;

    Circle(int x, int y, int r) {
        this.center = new Point(x, y);
        this.r = r;
    }
}
```

في هذا المثال، لا تمرر نقطة جاهزة إلى الدائرة، بل تنشأ داخلها. وهذا يعبر عن علاقة تركيب؛ لأن center جزء داخلي من الكائن Circle.

2. التجميع أو الارتباط الضعيف (Aggregation / Association)

التجميع هو علاقة احتواء أضعف من التركيب، ويكون فيها الكائن المحتوى قادراً على الوجود بشكل مستقل عن الكائن المحتوي. أما الارتباط (Association) فهو مفهوم أوسع يشير إلى وجود علاقة تعامل بين صنفين، سواء أكانت علاقة احتواء أم مجرد استخدام أو تواصل.

- آلية التجميع برمجيًا: يُنشأ الكائن المحتوى خارج الصنف، ثم يُمرر إلى الصنف الآخر عبر الباني أو عبر تابع، فيحتفظ الصنف بمرجعه فقط.

- مثال واقعي: يمكن تمثيل العلاقة بين السيارة Car والمحرك Engine كتجميع إذا كان المحرك كائنًا مستقلًا يمكن إنشاؤه خارج السيارة وتمريره إليها. في هذه الحالة، إذا اختفى كائن السيارة من البرنامج، لا يعني ذلك بالضرورة أن كائن المحرك لم يعد موجودًا.
مثال برمجي على ذلك:

```
class Engine {
    public void start() {
        System.out.println("Engine started");
    }
}

class Car {
    private Engine engine;

    Car(Engine engine) {
        this.engine = engine;
    }

    public void startCar() {
        engine.start();
    }
}
```

في هذا المثال، لم تنشئ السيارة المحرك داخلها، وإنما استقبلت مرجعًا إلى محرك موجود مسبقًا. لذلك تكون العلاقة أضعف من التركيب.

استخدامات الكلمة المفتاحية final

تستخدم الكلمة المفتاحية final في Java لفرض قيود تمنع التعديل أو إعادة التعريف، ويختلف أثرها بحسب موضع استخدامها.

1. مع المتغيرات (Variables):

تجعل المتغير غير قابل لإعادة الإسناد بعد إعطائه قيمة أولى. فإذا كان المتغير من نوع أولي مثل int أو float، تصبح قيمته ثابتة. أما إذا كان المتغير مرجعًا لكائن، فإن final تمنع تغيير المرجع نفسه إلى كائن آخر، لكنها لا تمنع بالضرورة تعديل محتوى الكائن إذا كان الكائن قابلاً للتعديل.

مثال:

```
final int x = 10;
```

هنا لا يمكن اسناد قيمة جديدة للمتحول x

2. مع التوابع (Methods):

تمنع الأصناف الأبناء من إعادة كتابة التابع باستخدام Override. وبذلك يبقى تنفيذ التابع كما هو في الصنف الأب.
مثال:

```
class Animal {
    public final void sleep() {
        System.out.println("Sleeping");
    }
}
```

في هذه الحالة، لا يستطيع أي صنف يرث من Animal أن يعيد كتابة التابع sleep.

3. مع الأصناف (Classes):

تمنع الوراثة من الصنف نهائياً. فإذا عُرّف الصنف باستخدام final، فلا يمكن لأي صنف آخر أن يكتب extends له.
مثال:

```
final class Animal{
}
```

في هذه الحالة لا يمكن إنشاء صنف يرث من Animal.
ومن الأمثلة المشهورة في Java الصنف String، إذ يُعرّف على أنه صنف نهائي final، ولذلك لا يمكن إنشاء صنف يرث منه.

*Code is like humor. When you
have to explain it, it's bad*

Cory House

